



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)

Y25 · 2026–2027 · Trimester 4 · Section 13 · Team 03

CAR RENTAL & MOBILITY PLATFORM

Review-2 · Literature Review

Team Members

2520080037 – R. Rohit

2520090027 – G. Bhupathi

2520090082 – S. Anjani Shreyaj

Contents of Review-2



01
Relevance to Title



02
Survey: Data Layer



03
Survey: Architecture



04
Survey: Search & Real-Time



05
Critical Analysis



06
Synthesis



07
Gap Analysis



08
Key Takeaways



09
References

Relevance to Project Title



Why This Literature

Our title, Car Rental & Mobility Platform, needs bookings, a vehicle catalog, live tracking, and recommendations. We reviewed work on each need.

Every source was chosen because it supports one design choice: data stores, services, consistency, search, or monitoring.

Together, the nine sources cover the whole technical stack of the platform.

Literature Areas Linked to Title



Polyglot Persistence [1]–[3]



Microservices [4], [5]



Saga Consistency [5], [6]



Vector Search [3], [7]



Real-Time & Tracing [8], [9]

Literature Survey 1: Data Layer

Ref	Study	Key Findings	Limitation
[1]	Scalable SQL and NoSQL Data Stores Cattell, 2010, ACM SIGMOD Record	Compares key-value, document, and scalable SQL stores on scalability, replication, and consistency.	No single store suits every workload; vector search and multi-store use are not covered.
[2]	NoSQL Distilled Sadalage & Fowler, 2012, Addison-Wesley	Introduces polyglot persistence: using different data stores for different needs in one application.	Mostly conceptual; no benchmark or reference build of consistency across stores.
[3]	Survey of Vector Database Management Systems Pan, Wang & Li, 2024, VLDB Journal	Reviews vector indexes and systems; native vector databases versus extended systems that add vectors to existing databases.	General survey, not tied to rental or mobility use; hybrid attribute plus vector queries remain hard.

Literature Survey 2: Architecture

Ref	Study	Key Findings	Limitation
[4]	Microservices: Yesterday, Today, and Tomorrow Dragoni et al., 2017, Springer	Traces how microservices evolved and why small independent services scale and deploy more easily.	Architecture-focused; gives limited guidance on data consistency across services.
[5]	Microservices Patterns Richardson, 2018, Manning	Catalogues patterns such as API gateway, database per service, and Saga for cross-service transactions.	Examples are generic and mainly Java-based; vector search and polyglot stores are not addressed.
[6]	Sagas Garcia-Molina & Salem, 1987, ACM SIGMOD	Splits a long transaction into sub-transactions with compensating actions, avoiding long-held locks.	Predates microservices; compensation logic must be designed for each application.

Literature Survey 3: Search & Real-Time

Ref	Study	Key Findings	Limitation
[7]	HNSW: Approximate Nearest Neighbor Search Malkov & Yashunin, 2020, IEEE TPAMI	Builds a multi-layer proximity graph that scales logarithmically and outperforms earlier vector-only methods.	Results are approximate and the index is memory-heavy; filtering by attributes needs extra handling.
[8]	The WebSocket Protocol (RFC 6455) Fette & Melnikov, 2011, IETF	Defines a two-way channel over one connection, so servers can push live updates to clients.	A protocol specification only; scaling many connections and delivery is left to the application.
[9]	Dapper: Distributed Tracing Infrastructure Sigelman et al., 2010, Google Tech Report	Shows low-overhead tracing across many services, helping find where requests slow down or fail.	Describes an internal Google system; open tools such as OpenTelemetry came later.

Critical Analysis



Common Ground

[1], [2], [4]: no single database or service design fits every workload.



Trade-offs

Scale versus consistency [1]; HNSW gains speed but is approximate [7].



Strengths

Sagas [6] avoid long locks;
WebSockets [8] push live data; tracing [9] aids debugging.



Weaknesses

Each paper studies one technique;
combining them adds complexity.

Synthesis of the Literature



One Integrated Design

Each source supports one design choice



Polyglot data layer: SQL, document, vector [1]–[3]



API gateway and microservices [4], [5]



Saga-based booking consistency [5], [6]



HNSW search [7], live updates [8], tracing [9]

Gap Analysis

Gap	Literature Offers	Gap in Reviewed Literature	How Our Project Addresses It
G1	Polyglot ideas and NoSQL comparisons [1], [2]	In the papers reviewed, few worked examples join SQL, document, and vector stores in one rental backend.	PostgreSQL, MongoDB, and pgvector working together in one car rental platform.
G2	Saga and microservice patterns [4]–[6]	Examples are generic, not shaped around a booking, payment, and inventory flow.	Saga with compensating steps for booking, payment, and inventory.
G3	HNSW and vector database surveys [3], [7]	Focus is on index algorithms, not recommendations kept beside transactional data.	pgvector with an HNSW index inside PostgreSQL for vehicle similarity.
G4	WebSockets [8] and tracing [9] studied alone	Rarely combined with booking flows and recommendations in one system.	Socket.io and Redis tracking, with OpenTelemetry, Prometheus, and Grafana.

Key Takeaways



The literature supports a polyglot, service-oriented design, but the reviewed work does not combine all its parts for car rental.



Literature Supports Us

Polyglot stores, microservices, Sagas, and HNSW are all established.



Gaps Remain

Integration, rental-specific consistency, and unified monitoring are open.



Our Contribution

An integrated, observable car rental backend that fills these gaps.

References

Ref	Citation
[1]	R. Cattell, "Scalable SQL and NoSQL data stores," ACM SIGMOD Record, vol. 39, no. 4, pp. 12–27, 2010.
[2]	P. J. Sadalage and M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley, 2012.
[3]	J. J. Pan, J. Wang, and G. Li, "Survey of vector database management systems," The VLDB Journal, vol. 33, pp. 1591–1615, 2024.
[4]	N. Dragoni et al., "Microservices: Yesterday, today, and tomorrow," in Present and Ulterior Software Engineering, Springer, 2017, pp. 195–216.
[5]	C. Richardson, Microservices Patterns. Manning Publications, 2018.
[6]	H. Garcia-Molina and K. Salem, "Sagas," in Proc. ACM SIGMOD Int. Conf. on Management of Data, 1987, pp. 249–259.
[7]	Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," IEEE TPAMI, vol. 42, no. 4, 2020.
[8]	I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, Internet Engineering Task Force, 2011.
[9]	B. H. Sigelman et al., "Dapper, a large-scale distributed systems tracing infrastructure," Google Technical Report, 2010.